

Accounts & keys

Every account, credential and approval needed to build, test and launch the Jigawa pilot — what each one is for, where it goes, how long it takes to get, and what it costs. Ordered by when it blocks the work.

00 How to hand over a secret safely

Read this once. It takes thirty seconds and it prevents the most common way small teams leak credentials.

NEVER PASTE A KEY INTO A CHAT, AN EMAIL, OR A WHATSAPP MESSAGE

Those all keep permanent copies you cannot delete — in transcripts, on servers, in backups, on other people's phones. A key that has been pasted into one is compromised from that moment, whether or not anyone misused it.

Instead: open `api/.env` on the machine and type the value in there. That file is on your computer only, and git is configured to refuse to upload it.

If a key does get exposed

Rotate it immediately and move on. Every service below lets you replace a key in under two minutes. Rotating early costs nothing; rotating after someone has drained a Paystack balance costs a great deal.

The best moment for this to happen is now — before any real athlete has registered, before any real money has moved. An exposure today is an inconvenience. The same exposure in week ten is an incident report.

Where secrets live, at each stage

- **While building** — `api/.env` on this machine. Never committed.
- **Staging and production** — the hosting provider's secret store, entered once through their dashboard. Never in a file, never in the repository.
- **Test versus live keys** — Paystack and Termii both issue test credentials. Use those everywhere except production. The application refuses to start in production with a test payment key, so a mix-up fails loudly instead of quietly taking fake money.

01 Needed now — to build and test

Nothing works without these. All are free at this stage and available the same day.

ACCOUNT	WHAT TO GET	GOES INTO	WHAT IT DOES
Supabase BLOCKING	Database password <i>Direct connection string</i> (port 5432, not the pooler)	DATABASE_URL_MIGRATE DATABASE_URL_APP DATABASE_URL_MONEY DATABASE_URL_READER	The database. Holds every athlete record, every KUID, every ledger entry. I generate the four role passwords myself — you only supply the one admin connection string.
GitHub BLOCKING	Two private repositories Personal access token or SSH key	Git remote, not a file	Where the code lives. Must be private — <code>docs/</code> contains the CEO memo, the GDOC and the build plan.
Paystack TEST ONLY	Test secret key <code>sk_test_...</code>	PAYSTACK_SECRET_KEY	Lets the whole payment flow be built and tested with fake cards. Available immediately on signup — the business verification that takes weeks is only needed for real money.
Termii TEST ONLY	API key	TERMII_API_KEY	Sends the SMS confirmation code. A small credit is enough for testing. The sender ID approval that takes days is only needed for launch.

ALREADY GENERATED, NOTHING FOR YOU TO DO

`SECRET_KEY` and `QR_SECRET` were generated on the machine and written straight into `api/.env`. They never appeared on screen and never passed through a message. `QR_SECRET` is the one that signs every printed athlete card, so it is the single most important value in the file.

02 Needed soon — these have lead times

None of them block writing code. All of them block going live, and two of them cannot be hurried.

ACCOUNT	WHAT TO GET	LEAD TIME	WHY IT MATTERS, AND THE TRAP
Paystack live	Business verification Live secret key <code>sk_live_...</code> Webhook URL registered	1–3 weeks	The only way real money enters the system. Requires current CAC business registration and a corporate bank account. If the CAC registration is not current, that becomes the longest single task on the whole project — check this week, not in week six.
Termii live	Sender ID approved (e.g. <code>KAFRIADA</code>)	3–10 days	Without it, no confirmation codes; without codes, nobody can register. Register a numeric fallback route at the same time so a rejected sender ID delays branding rather than the launch.
Domain	e.g. <code>kafriada.ng</code> Registrar login	1–3 days	Every printed QR card points at this address <i>permanently</i> . Cards cannot be reprinted once handed out, so the domain must be one you will keep for years. Choose carefully and set auto-renew.
Cloudflare	Account, zone for the domain API token (DNS edit only)	Same day	Caches the public profile pages at their Lagos point of presence, so a match-day scan is answered in Nigeria and never reaches the server. Also the firewall.
Cloudflare R2	Bucket (private) Access key ID + secret Account ID	Same day	Stores athlete photos and identity documents. The bucket must be private — a public bucket means anyone with a URL sees a photo without paying, and the ₦2,500 product is given away.
Hosting	Render or Railway account Frankfurt region Payment card	Same day	Runs the application and the background worker. Frankfurt because Nigeria reaches it in roughly 90–130ms, against nearly double that to America.

Before launch — small, but do not skip

Cheap or free, and each one is something you will wish you had the first time it is needed.

ACCOUNT	WHAT TO GET	GOES INTO	WHAT IT DOES
Sentry	Project DSN	<code>SENTRY_DSN</code>	Tells you something broke, with the exact cause. Working alone, this is what watches while you sleep. Free tier is enough.
Resend or SES	API key Domain verified (SPF, DKIM)	<code>RESEND_API_KEY</code>	Receipts and admin notifications. Domain verification takes a day and must be done before launch or the mail lands in spam.
Uptime monitor	Free account (UptimeRobot or similar)	Dashboard only	Alerts you when the site is down. Without it, the first person who knows is a coordinator standing in a field with 40 athletes.
NDPR compliance	A named person and a budget line	Nothing technical	Nigerian data protection filing obligations begin above 2,000 data subjects in twelve months. The pilot targets 5,000. Still unassigned.

What this costs each month

Approximate, and worth checking against each provider's current pricing page.

ITEM	WHILE BUILDING	LIVE PILOT
Supabase database	Free	~\$25 — free projects pause after about a week idle, which is fine now and unacceptable with real users
Supabase point-in-time backups	—	~\$100 add-on. Without it you get one backup a day, so a bad afternoon can lose a morning of registrations
Hosting — app + worker	Free tier	~\$32
Cloudflare	Free	Free, or \$20 for managed firewall rules
R2 storage	Free	~\$0 at pilot volume
Termii SMS	~\$5 credit	~\$25–40, scales with signups
Sentry, Resend, monitor	Free	Free
Total	~\$5	~\$180–220

THE BACKUP LINE IS THE ONE TO THINK ABOUT

Point-in-time recovery is the difference between losing five minutes of registrations and losing a whole day of them. It is also the single largest line in the table. If the budget will not carry it at launch, the honest alternative is to accept a 24-hour recovery point, write that down as a known risk, and tell the CEO — rather than assume a protection that was never bought.

Using Supabase Auth — what it changes

A real saving, and three specific costs. Worth five minutes before it is built on.

Supabase Auth handles sign-up, sign-in, confirmation codes and password reset. Building all of that by hand is roughly **four to five days of solo work**, so the appeal is genuine and the instinct is sound.

Three things about this system make it a closer call than it looks. None of them is a reason to refuse — they are things to decide deliberately rather than discover in week six.

1. Coordinators share phones, and Supabase logins cannot be cut off instantly

Supabase issues the user a pass that the phone keeps and presents on each request. The server does not check a list each time — it just reads the pass. That is fast, and it is why the pass **stays valid until it expires**, typically an hour, even after you remove someone's access.

For an athlete that is fine. For an **LGA coordinator handling cash**, on a phone shared with other staff, it is the difference between "access ended now" and "access ends within the hour". Our own design keeps a list of live sessions in the database, so revoking one takes effect on the very next request, and staff sessions expire after 30 minutes idle.

2. Termii is not one of Supabase's SMS providers

Supabase sends confirmation codes through Twilio, Vonage and similar. **Termii is not on that list**. Two ways round it, and both cost something:

- **Switch to Twilio.** Works immediately, but Nigerian SMS through an international provider is typically several times the price per message, and the budget and the sender-ID work already assume Termii.
- **Keep Termii via a custom hook.** Supabase generates the code and calls our server to deliver it. Supported, and it means we still write the delivery code — so part of the saving disappears.

3. Sign-up must work with JavaScript switched off

Supabase Auth is normally driven by JavaScript in the browser. Opera Mini in proxy mode — common in northern Nigeria — barely runs JavaScript, and registration is the one flow that absolutely cannot fail there. Solvable: our server calls Supabase Auth's API on the user's behalf, and the browser only ever submits a plain form. But it has to be built that way on purpose.

RECOMMENDATION — TAKE THE SAVING, KEEP THE CONTROL

Use Supabase Auth for what it is excellent at, and keep our own session cookie.

Supabase stores the passwords, generates and checks the confirmation codes, and handles password reset. Our server talks to it behind the scenes, and once a person is confirmed, **we issue our own session** — the one recorded in the database, revocable instantly, expiring after 30 minutes idle for staff.

The phone never receives a Supabase pass, so nothing is stored in a browser that we cannot cancel. Registration stays a plain form that works without JavaScript. Roles and scope stay ours, because Supabase has no concept of "this coordinator, this LGA only".

You keep most of the time saving. You keep instant revocation, which matters specifically because coordinators handle cash on shared phones.

WHAT CHANGES IN THE CREDENTIALS LIST

- **Add** the Supabase project URL and the service role key to `api/.env` — the server uses them to talk to Supabase Auth.
- The **anon key is not needed** under this approach. It exists for browsers talking to Supabase directly, and in this design no browser ever does.
- **The service role key must never reach the browser or the frontend repository.** It bypasses every security rule in the project. Server-side only, always.
- `SECRET_KEY` and `QR_SECRET` stay exactly as they are — those are ours and unrelated.

IF YOU WOULD RATHER USE SUPABASE AUTH FULLY, IN THE BROWSER

That is a legitimate choice and it is faster still. Two consequences to accept in writing rather than by default: **a removed coordinator keeps access until their pass expires**, and the registration screens need a separate no-JavaScript path built anyway. Tell me and I will build it that way — I would want it recorded as a decision, not arrived at silently.

06 Two repositories — one thing to handle

Front end and back end now live separately. Workable, with one safeguard to add.

The two repositories are `kafriadadev_backend` and `kafriadadev_frontend`. Splitting them is a normal choice and there is nothing wrong with it.

It does remove one safety net that a single repository gave us for free. The front end and the back end have to agree on the exact shape of every message between them — every field name, every type. When they share a repository, a change that breaks that agreement **fails the build immediately, in the same commit**. Split apart, the back end can change and the front end keeps compiling happily against an agreement that no longer holds. The failure then shows up in a browser, in front of a coordinator, rather than in a build log.

THE SAFEGUARD

The back end publishes its agreed shape as a file on every change. The front end's build fetches that file and checks itself against it, and **fails if it has fallen behind**. Half a day to set up, and it restores the guarantee that the split gave away.

Worth doing in Stage 0, while there is almost nothing to keep in sync — retrofitting it once dozens of screens exist is a much larger job.

BOTH REPOSITORIES MUST BE PRIVATE

The back end carries `docs/` — the CEO memo, the 244-page GDOC, the build plan with its costs and estimates. None of that should be public, and a repository made public by accident cannot be made private again in any meaningful sense: it has already been cloned and indexed.

07 What happens next

In order. Only the first two need you.

- 1 **Rotate the exposed keys.** Supabase → Project Settings → Database → Reset database password. Then Project Settings → API → JWT Settings → Generate new secret. Two minutes.
- 2 **Put the new database password into** `api/.env` on the machine, in the `DATABASE_URL_MIGRATE` line. Not into a message.
- 3 **Decide the sign-in question** in section 05 — the recommended middle path, or full Supabase Auth in the browser.
- 4 I create the four database roles, generate their passwords, and apply the privilege boundary.
- 5 I run the migration against the real database and then **prove the audit log cannot be tampered with** — the security guarantee that is currently written but not yet demonstrated.
- 6 I push to the two private repositories and wire up the build checks.
- 7 Stage 1 begins: registration, KUID, the public profile and the printed QR card.

AND THIS WEEK, SEPARATELY FROM THE CODE

Check that **KowaGuru's CAC registration is current**, then start Paystack business verification and the Termii sender ID. Neither can be hurried, both block the launch rather than the build, and the CAC question is the one that can quietly become the longest task on the entire project.